

Decoding lab (mysecret) in Kylin Desktop OS V10

Target

1. Decoding lab: understanding a Secret Message You have just intercepted an encoded message in kylin desktop os v10

破解秘密消息

Requirement

- The source code file of mysecret.cpp cannot be changed, and the secret can only be cracked by providing specific parameters during runtime.

不能改变mysecret.cpp

- 只能在Kylin Desktop OS v10下完成此次任务

Tools

Install GCC Software Colletion

```
sudo apt-get install build-essential
```

How to use GCC

- [gcc and make](#)

gdb

- gdb mysecret
 - b main
 - run 123 456
 - ...

mysecret反汇编

- in gdb
 - disass main

(gdb) disass main

Dump of assembler code for function main(int, char**):

```
0x080493cf <+0>:      endbr32
0x080493d3 <+4>:      lea    0x4(%esp),%ecx
0x080493d7 <+8>:      and    $0xffffffff0,%esp
0x080493da <+11>:     pushl  -0x4(%ecx)
0x080493dd <+14>:     push  %ebp
0x080493de <+15>:     mov    %esp,%ebp
0x080493e0 <+17>:     push  %ebx
0x080493e1 <+18>:     push  %ecx
0x080493e2 <+19>:     sub    $0x40,%esp
0x080493e5 <+22>:     mov    %ecx,%ebx
0x080493e7 <+24>:     mov    0x4(%ebx),%eax
0x080493ea <+27>:     mov    %eax,-0x3c(%ebp)
0x080493ed <+30>:     mov    %gs:0x14,%eax
0x080493f3 <+36>:     mov    %eax,-0xc(%ebp)
0x080493f6 <+39>:     xor    %eax,%eax
0x080493f8 <+41>:     movl   $0x1,-0x30(%ebp)
0x080493ff <+48>:     movl   $0x2,-0x1c(%ebp)
0x08049406 <+55>:     movl   $0x3,-0x18(%ebp)
0x0804940d <+62>:     movl   $0x0,-0x2c(%ebp)
0x08049414 <+69>:     movl   $0x0,-0x28(%ebp)
0x0804941b <+76>:     movl   $0x0,-0x24(%ebp)
0x08049422 <+83>:     movl   $0x0,-0x20(%ebp)
--Type <RET> for more, q to quit, c to continue without paging--c
0x08049429 <+90>:     movl   $0x0,-0x20(%ebp)
0x08049430 <+97>:     mov    -0x20(%ebp),%eax
0x08049433 <+100>:    mov    %eax,-0x24(%ebp)
0x08049436 <+103>:    cmpl   $0x2,(%ebx)
0x08049439 <+106>:    jg     0x804944c <main(int, char**)+125>
0x0804943b <+108>:    mov    -0x3c(%ebp),%eax
0x0804943e <+111>:    mov    (%eax),%eax
0x08049440 <+113>:    sub    $0xc,%esp
0x08049443 <+116>:    push  %eax
0x08049444 <+117>:    call  0x8049256 <usage_and_exit(char*)>
0x08049449 <+122>:    add    $0x10,%esp
0x0804944c <+125>:    mov    -0x3c(%ebp),%eax
0x0804944f <+128>:    add    $0x4,%eax
0x08049452 <+131>:    mov    (%eax),%eax
0x08049454 <+133>:    sub    $0x4,%esp
0x08049457 <+136>:    push  $0x0
0x08049459 <+138>:    push  $0x0
0x0804945b <+140>:    push  %eax
```

```

0x0804945c <+141>:    call    0x8049130 <strtol@plt>
0x08049461 <+146>:    add     $0x10,%esp
0x08049464 <+149>:    mov     %eax,-0x2c(%ebp)
0x08049467 <+152>:    mov     -0x3c(%ebp),%eax
0x0804946a <+155>:    add     $0x8,%eax
0x0804946d <+158>:    mov     (%eax),%eax
0x0804946f <+160>:    sub     $0x4,%esp
0x08049472 <+163>:    push    $0x0
0x08049474 <+165>:    push    $0x0
0x08049476 <+167>:    push    %eax
0x08049477 <+168>:    call    0x8049130 <strtol@plt>
0x0804947c <+173>:    add     $0x10,%esp
0x0804947f <+176>:    mov     %eax,-0x28(%ebp)
0x08049482 <+179>:    cmpl    $0x3,(%ebx)
0x08049485 <+182>:    jle     0x80494a2 <main(int, char**)+211>
0x08049487 <+184>:    mov     -0x3c(%ebp),%eax
0x0804948a <+187>:    add     $0xc,%eax
0x0804948d <+190>:    mov     (%eax),%eax
0x0804948f <+192>:    sub     $0x4,%esp
0x08049492 <+195>:    push    $0x0
0x08049494 <+197>:    push    $0x0
0x08049496 <+199>:    push    %eax
0x08049497 <+200>:    call    0x8049130 <strtol@plt>
0x0804949c <+205>:    add     $0x10,%esp
0x0804949f <+208>:    mov     %eax,-0x24(%ebp)
=> 0x080494a2 <+211>:    cmpl    $0x4,(%ebx)
0x080494a5 <+214>:    jle     0x80494c2 <main(int, char**)+243>
0x080494a7 <+216>:    mov     -0x3c(%ebp),%eax
0x080494aa <+219>:    add     $0x10,%eax
0x080494ad <+222>:    mov     (%eax),%eax
0x080494af <+224>:    sub     $0x4,%esp
0x080494b2 <+227>:    push    $0x0
0x080494b4 <+229>:    push    $0x0
0x080494b6 <+231>:    push    %eax
0x080494b7 <+232>:    call    0x8049130 <strtol@plt>
0x080494bc <+237>:    add     $0x10,%esp
0x080494bf <+240>:    mov     %eax,-0x20(%ebp)
0x080494c2 <+243>:    sub     $0x8,%esp
0x080494c5 <+246>:    lea     -0x28(%ebp),%eax
0x080494c8 <+249>:    push    %eax
0x080494c9 <+250>:    lea     -0x2c(%ebp),%eax
0x080494cc <+253>:    push    %eax
0x080494cd <+254>:    call    0x8049298 <process_keys12(int*, int*)>

```

```

0x080494d2 <+259>:      add     $0x10,%esp
0x080494d5 <+262>:      lea     -0x30(%ebp),%eax
0x080494d8 <+265>:      movzbl (%eax),%eax
0x080494db <+268>:      movsbl %al,%eax
0x080494de <+271>:      mov     %eax,-0x1c(%ebp)
0x080494e1 <+274>:      lea     -0x30(%ebp),%eax
0x080494e4 <+277>:      add     $0x1,%eax
0x080494e7 <+280>:      movzbl (%eax),%eax
0x080494ea <+283>:      movsbl %al,%eax
0x080494ed <+286>:      mov     %eax,-0x18(%ebp)
0x080494f0 <+289>:      mov     -0x24(%ebp),%eax
0x080494f3 <+292>:      test    %eax,%eax
0x080494f5 <+294>:      je      0x8049511 <main(int, char**)+322>
0x080494f7 <+296>:      mov     -0x20(%ebp),%eax
0x080494fa <+299>:      test    %eax,%eax
0x080494fc <+301>:      je      0x8049511 <main(int, char**)+322>
0x080494fe <+303>:      sub     $0x8,%esp
0x08049501 <+306>:      lea     -0x20(%ebp),%eax
0x08049504 <+309>:      push    %eax
0x08049505 <+310>:      lea     -0x24(%ebp),%eax
0x08049508 <+313>:      push    %eax
0x08049509 <+314>:      call    0x80492ba <process_keys34(int*, int*)>
0x0804950e <+319>:      add     $0x10,%esp
0x08049511 <+322>:      sub     $0x8,%esp
0x08049514 <+325>:      pushl   -0x18(%ebp)
0x08049517 <+328>:      pushl   -0x1c(%ebp)
0x0804951a <+331>:      call    0x80492eb <extract_message1(int, int)>
0x0804951f <+336>:      add     $0x10,%esp
0x08049522 <+339>:      mov     %eax,-0x14(%ebp)
0x08049525 <+342>:      mov     -0x14(%ebp),%eax
0x08049528 <+345>:      movzbl (%eax),%eax
0x0804952b <+348>:      test    %al,%al
0x0804952d <+350>:      jne     0x8049566 <main(int, char**)+407>
0x0804952f <+352>:      sub     $0x8,%esp
0x08049532 <+355>:      lea     -0x20(%ebp),%eax
0x08049535 <+358>:      push    %eax
0x08049536 <+359>:      lea     -0x24(%ebp),%eax
0x08049539 <+362>:      push    %eax
0x0804953a <+363>:      call    0x80492ba <process_keys34(int*, int*)>
0x0804953f <+368>:      add     $0x10,%esp
0x08049542 <+371>:      sub     $0x8,%esp
0x08049545 <+374>:      pushl   -0x18(%ebp)
0x08049548 <+377>:      pushl   -0x1c(%ebp)

```

```

0x0804954b <+380>:      call    0x8049375 <extract_message2(int, int)>
0x08049550 <+385>:      add     $0x10,%esp
0x08049553 <+388>:      mov     %eax,-0x10(%ebp)
0x08049556 <+391>:      sub     $0xc,%esp
0x08049559 <+394>:      pushl   -0x10(%ebp)
0x0804955c <+397>:      call    0x80490f0 <puts@plt>
0x08049561 <+402>:      add     $0x10,%esp
0x08049564 <+405>:      jmp     0x8049574 <main(int, char**)+421>
0x08049566 <+407>:      sub     $0xc,%esp
0x08049569 <+410>:      pushl   -0x14(%ebp)
0x0804956c <+413>:      call    0x80490f0 <puts@plt>
0x08049571 <+418>:      add     $0x10,%esp
0x08049574 <+421>:      sub     $0xc,%esp
0x08049577 <+424>:      push    $0x804a027
0x0804957c <+429>:      call    0x80490c0 <printf@plt>
0x08049581 <+434>:      add     $0x10,%esp
0x08049584 <+437>:      call    0x80490d0 <getchar@plt>
0x08049589 <+442>:      mov     $0x0,%eax
0x0804958e <+447>:      mov     -0xc(%ebp),%edx
0x08049591 <+450>:      xor     %gs:0x14,%edx
0x08049598 <+457>:      je      0x804959f <main(int, char**)+464>
0x0804959a <+459>:      call    0x80490e0 <__stack_chk_fail@plt>
0x0804959f <+464>:      lea     -0x8(%ebp),%esp
0x080495a2 <+467>:      pop     %ecx
0x080495a3 <+468>:      pop     %ebx
0x080495a4 <+469>:      pop     %ebp
0x080495a5 <+470>:      lea     -0x4(%ecx),%esp
0x080495a8 <+473>:      ret

```

End of assembler dump.

(gdb)

How to do

Step0: cat mysecret.cpp

```

#include <stdio.h>
#include <stdlib.h>

int prologue [] = {
    0x5920453A, 0x54756F0A, 0x6F6F470A, 0x21643A6F,
    0x6E617920, 0x680A6474, 0x6F697661, 0x20646E69,
    0x63636363, 0x63636363, 0x72464663, 0x6F6D6F72,
    0x63636363, 0x63636363, 0x72464663, 0x6F6D6F72,
    0x2C336573, 0x7420346E, 0x20216F74, 0x726F5966,
    0x7565636F, 0x20206120, 0x6C616763, 0x74206C6F,
    0x20206F74, 0x74786565, 0x65617276, 0x32727463,
    0x594E2020, 0x206F776F, 0x79727574, 0x4563200A
};

int data [] = {
    0x63636363, 0x63636363, 0x72464663, 0x6F6D6F72,
    0x466D203A, 0x65693A72, 0x43646E20, 0x6F54540A,
    0x5920453A, 0x54756F0A, 0x6F6F470A, 0x21643A6F,
    0x594E2020, 0x206F776F, 0x79727574, 0x4563200A,
    0x6F786F68, 0x6E696373, 0x6C206765, 0x796C656B,
    0x2C336573, 0x7420346E, 0x20216F74, 0x726F5966,
    0x7565636F, 0x20206120, 0x6C616763, 0x74206C6F,
    0x20206F74, 0x74786565, 0x65617276, 0x32727463,
    0x6E617920, 0x680A6474, 0x6F697661, 0x20646E69,
    0x21687467, 0x63002065, 0x6C6C7861, 0x78742078,
    0x6578206F, 0x72747878, 0x78636178, 0x00783174
};

int epilogue [] = {
    0x594E2020, 0x206F776F, 0x79727574, 0x4563200A,
    0x6E617920, 0x680A6474, 0x6F697661, 0x20646E69,
    0x7565636F, 0x20206120, 0x6C616763, 0x74206C6F,
    0x2C336573, 0x7420346E, 0x20216F74, 0x726F5966,
    0x20206F74, 0x74786565, 0x65617276, 0x32727463
};

/*message[100]: too small */
char message[200];

void usage_and_exit(char * program_name) {
    fprintf(stderr, "USAGE: %s key1 key2 key3 key4\n", program_name);
    printf("Press any key to exit...");
    getchar();
}

```

```

        exit(1);
    }

void process_keys12 (int * key1, int * key2) {

    *((int *) (key1 + *key1)) = *key2;
}

void process_keys34 (int * key3, int * key4) {

    (((int *)&key3) + *key3) += *key4;
}

char * extract_message1(int start, int stride) {
    int i, j, k;
    int done = 0;

    for (i = 0, j = start + 1; ! done; j++) {
        for (k = 1; k < stride; k++, j++, i++) {

            if (((char *) data) + j) == '\0') {
                done = 1;
                break;
            }

            message[i] = (((char *) data) + j);
        }
    }
    message[i] = '\0';
    return message;
}

char * extract_message2(int start, int stride) {
    int i, j;

    for (i = 0, j = start;
        (((char *) data) + j) != '\0';
        i++, j += stride)
    {
        message[i] = (((char *) data) + j);
    }
    message[i] = '\0';
    return message;
}

```



```
}
```

```
int main (int argc, char *argv[])
```

```
{
```

```
    int dummy = 1;
```

```
    int start = 2, stride = 3;
```

```
    int key1 = 0, key2 = 0, key3 = 0, key4 = 0;
```

```
    char * msg1, * msg2;
```

```
    key3 = key4 = 0;
```

```
    if (argc < 3) {
```

```
        usage_and_exit(argv[0]);
```

```
    }
```

```
    key1 = strtol(argv[1], NULL, 0);
```

```
    key2 = strtol(argv[2], NULL, 0);
```

```
    if (argc > 3) key3 = strtol(argv[3], NULL, 0);
```

```
    if (argc > 4) key4 = strtol(argv[4], NULL, 0);
```

```
    process_keys12(&key1, &key2);
```

```
    start = (int)*(((char *) &dummy));
```

```
    stride = (int)*(((char *) &dummy) + 1));
```

```
    if (key3 != 0 && key4 != 0) {
```

```
        process_keys34(&key3, &key4);
```

```
    }
```

```
    msg1 = extract_message1(start, stride);
```

```
    if (*msg1 == '\0') {
```

```
        process_keys34(&key3, &key4);
```

```
        msg2 = extract_message2(start, stride);
```

```
        printf("%s\n", msg2);
```

```
    }
```

```
    else {
```

```
        printf("%s\n", msg1);
```

```
    }
```

```
    printf("Press any key to exit...");
```

```
    getchar();
```

```
    return 0;
```

```
}
```

- sha256sum mysecret.cpp

```
e055e860a40ca8ff8fc13ff62e6844449c44dfad4e0200cb7ef78b95bf51e1a1 mysecret.cpp
```

Step1

- 在Kylin Desktop OS v10下编译mysecret.cpp，生成mysecret可执行文件
i. 编译为32位代码，且带调试信息"-g"

```
gcc -m32 -g -o mysecret mysecret.cpp
```

Step2

共两部分实验:

Step2.1

运行mysecret，尝试破解秘密消息第一部分

```
.\mysecret 123 456
```

albert@kyosdv10: ~/workspace/progs_c/os_lab_secret

— □ ×

文件(F) 编辑(E) 视图(V) 搜索(S) 终端(T) 帮助(H)

albert@kyosdv10:~/workspace/progs_c/os_lab_secret\$./mysecret X Y

From: Friend

To: You

Good! Now try choosing keys3,4 to force a call to extract2 and
avoid the call to extract1

Press any key to exit...

Step2.2

运行mysecret，尝试破解秘密消息第二部分

```
.\mysecret 123 456 78 90
```

```
albert@kyosdv10: ~/workspace/progs_c/os_lab_secret
文件(F) 编辑(E) 视图(V) 搜索(S) 终端(T) 帮助(H)
albert@kyosdv10:~/workspace/progs_c/os_lab_secret$ ./mysecret X Y
From: Friend
To: You
Good! Now try choosing keys3,4 to force a call to extract2 and
avoid the call to extract1
Press any key to exit...
albert@kyosdv10:~/workspace/progs_c/os_lab_secret$ ./mysecret X Y M N
From: Friend
To: You
Good! Now try choosing keys3,4 to force a call to extract2 and
avoid the call to extract1
Press any key to exit...
albert@kyosdv10:~/workspace/progs_c/os_lab_secret$
```

上图似乎有误导啊!!!

Task

- 实现上述实例，并给出原理分析和实际关键步骤
- 撰写实验报告

参考接下来的文档

Exercise: Decoding Lab

Decoding Lab:

Understanding a Secret Message You have just intercepted an encoded message. The message is a sequence of bits which reads as follows in hexadecimal:

```
63636363636363724646636F6D6F72
466D203A65693A7243646E206F54540A
5920453A54756F0A6F6F470A21643A6F
594E2020206F776F797275744563200A
6F786F686E6963736C206765796C656B
2C3365737420346E20216F74726F5966
7565636F202061206C61676374206C6F
20206F74747865656561727632727463
6E617920680A64746F69766120646E69
21687467630020656C6C786178742078
6578206F727478787863617800783174
```

You have no idea how to decode it, but you know that your grade depends on it, so you are willing to do anything to extract the message.

Fortunately, one of your many agents on the field has stolen the source code for the decoder. This agent (007) has put the code and the message in the file `secret.cpp`, which you can download from the laboratory of your technical staff (Q). Q has noticed that the decoder takes four integers as arguments. Executing the decoder with various arguments seems to either crash the program or produce unintelligible output. It seems that the correct four integers have to be chosen in order for the program to produce the decoded message. These four integers are the "secret keys." 007 has been unable to find the keys, but from the desk of the encrypting personnel he was able to cunningly retrieve the first five characters of the unencoded message. These characters are: `From:Assignment` Your assignment is to decode the message, and find the keys. Reminders This exercise is not extremely difficult. However, the strategy of trying things until something works will be ineffective. Try to understand the material in the course, particularly the following:

- Memory contains nothing but bits. Bits are interpreted as integers, characters, or instructions by the compiler, but they have no intrinsic type in memory.
- The compiler can be strong-armed into interpreting integers as characters, or even as instructions, and vice versa.
- Every group of 8 bits (a byte) has an address.
- A pointer in C is merely a stored memory address.
- The activation records for each function call are all together in memory, and they are organized in a stack that grows downwards and shrinks upwards on function calls and returns respectively.
-

The return address of one function as well as the addresses of all

of its local variables are allocated within one activation record.

StrategyThe designers of this decoder weren't very good. They made it possible for us to attack the keys in two independent parts. Try to break the first two keys first, and do not try to break the third and fourth keys until you have succeeded with the first two. You can do the first part by specifying only two integer arguments when you execute the decoder. If you get the first and second keys right, a message that starts with From: will appear. This message is not the true message, but a decoy. It is useful, however, to let you know that you have indeed broken the first two keys. In breaking the first two keys, realize that the function process_keys12 must be somehow changing the value of the dummy variable. This must be so, because the variables start and stride control the extraction of the message, and they are calculated from the value of dummy. In breaking the third and fourth keys, try to get the code to invoke extract_message2 instead of extract_message1. This modification must somehow be controlled from within the function process_keys34. FilesWhen you are done, write a brief report that includes at least the following:

1. The secret message.
2. The secret keys.
3. One paragraph describing, in your own prose, what process_keys12 does. For example, you might say that it modifies a specific program variable.
4. The meaning of the first two keys in terms of variables and addresses in the decoder program. For example, you might describe key2 by saying that its X-Y bits contain the value to which variable start is set. Or you might describe key1 by saying, for example, that it must be set equal to the number of memory addresses separating the address of two specific variables. These are only examples.
5. One paragraph describing, in your own prose, what process_keys34 does.
6. One paragraph describing the line of source code that is executed when the first call to process_keys34 returns.
7. The meaning of the third and fourth keys in terms of variables and addresses in the decoder program.

Be precise, clear, and brief in each of the points above. Your report should not, in any case, be longer than one page. Do not get frustrated if this takes a little longer than you expected: brief and clear text often requires more time to write than rambling prose. Your teacher can tell you what word processors you may use to write your report. Chances are that you can write your report in a number of formats, and for simplicity's sake, you might even want to write it using Notepad. Enjoy!

You have just intercepted an encoded message. The message is a sequence of bits which reads as follows in hexadecimal:

Decoding Lab: Understanding a Secret Message
6363636363636363724646636F6D6F72
466D203A65693A7243646E206F54540A

5920453A54756F0A6F6F470A21643A6F
594E2020206F776F797275744563200A
6F786F686E6963736C206765796C656B
2C3365737420346E20216F74726F5966
7565636F202061206C61676374206C6F
20206F74747865656561727632727463
6E617920680A64746F69766120646E69
21687467630020656C6C786178742078
6578206F727478787863617800783174

You have no idea how to decode it, but you know that your grade depends on it, so you are willing to do anything to extract the message. Fortunately, one of your many agents on the field has stolen the source code for the decoder. This agent (007) has put the code and the message in the file `secret.cpp`, which you can download from the laboratory of your technical staff (Q). Q has noticed that the decoder takes four integers as arguments. Executing the decoder with various arguments seems to either crash the program or produce unintelligible output. It seems that the correct four integers have to be chosen in order for the program to produce the decoded message. These four integers are the "secret keys." 007 has been unable to find the keys, but from the desk of the encrypting personnel he was able to cunningly retrieve the first five characters of the unencoded message. These characters are: `From:AssignmentYour assignment is to decode the message, and find the keys. RemindersThis exercise is not extremely difficult. However, the strategy of trying things until something works will be ineffective. Try to understand the material in the course, particularly the following:`

- Memory contains nothing but bits. Bits are interpreted as integers, characters, or instructions by the compiler, but they have no intrinsic type in memory.
- The compiler can be strong-armed into interpreting integers as characters, or even as instructions, and vice versa.
- Every group of 8 bits (a byte) has an address.
- A pointer in C is merely a stored memory address.
- The activation records for each function call are all together in memory, and they are organized in a stack that grows downwards and shrinks upwards on function calls and returns respectively.
-

The return address of one function as well as the addresses of all of its local variables are allocated within one activation record. StrategyThe designers of this decoder weren't very good. They made it possible for us to attack the keys in two independent parts. Try to break the first two keys first, and do not try to break the third and fourth keys until you have succeeded with the first two. You can do the first part by specifying only two integer arguments when you execute the decoder. If you get the first and second keys right, a message that starts with `From:` will appear. This message is not the true message, but a decoy. It is useful, however, to let you know that you have indeed broken the first

two keys. In breaking the first two keys, realize that the function `process_keys12` must be somehow changing the value of the dummy variable. This must be so, because the variables `start` and `stride` control the extraction of the message, and they are calculated from the value of `dummy`. In breaking the third and fourth keys, try to get the code to invoke `extract_message2` instead of `extract_message1`. This modification must somehow be controlled from within the function `process_keys34`. Files

When you are done, write a brief report that includes at least the following:

1. The secret message.
2. The secret keys.
3. One paragraph describing, in your own prose, what `process_keys12` does. For example, you might say that it modifies a specific program variable.
4. The meaning of the first two keys in terms of variables and addresses in the decoder program. For example, you might describe `key2` by saying that its X-Y bits contain the value to which variable `start` is set. Or you might describe `key1` by saying, for example, that it must be set equal to the number of memory addresses separating the address of two specific variables. These are only examples.
5. One paragraph describing, in your own prose, what `process_keys34` does.
6. One paragraph describing the line of source code that is executed when the first call to `process_keys34` returns.
7. The meaning of the third and fourth keys in terms of variables and addresses in the decoder program.

Be precise, clear, and brief in each of the points above. Your report should not, in any case, be longer than one page. Do not get frustrated if this takes a little longer than you expected: brief and clear text often requires more time to write than rambling prose. Your teacher can tell you what word processors you may use to write your report. Chances are that you can write your report in a number of formats, and for simplicity's sake, you might even want to write it using Notepad. Enjoy!

End.